

Lab 15 — A Calculator, Stage by Stage

DSA27 Lab Manual · Week 15 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 15

Contents

Part 0 — Before you start	2
Part 1 — tokenize: characters into tokens	4
Part 2 — The grammar on paper	6
Part 3 — The parser: parse, expr, term, factor	7
Part 4 — Break it: two grammars that look right	9
Part 5 — evaluate and calculate	10
Part 6 — See it	12
Part 7 — Measure it	13
Part 8 — Exercises at a glance	14
Part 9 — Take-home practice (not graded)	15
Part 10 — Course wrap-up and exam preparation	15
Summary	16
Answers to the checkpoints	17

How to use this lab. The routine you know: read the lecture section named at the top of each part, **draw before you code** — this week that means a tree on paper for every expression — and write your prediction at each **Checkpoint** before you look at the answers at the end. Build `dsa/translation.py` one stage at a time and run the tests of that stage before moving on: a parser tested on top of a broken lexer fails in ways that look like parser bugs. Parts 1–4 are the session; Parts 5–7 are for home. This is the last lab of the course, so it ends with a wrap-up and a plan for the final exam instead of a bridge to next week.

Duration	One 2-hour lab session, plus about 3 hours at home
----------	--

You will write

dsa/translation.py — tokenize, Parser.parse, Parser.expr, Parser.term, Parser.factor, evaluate, calculate (evaluate_postfix is your Week 6 work)

Graded by

tests/test_translation.py (55 tests)

Connects to

Lecture 15 — The principles of language translation; Lecture 06 (your Stack, postfix, shunting-yard); Lecture 03 (recursion); Lecture 11 (trees and post-order)

What you will be able to do

1. write a lexer as one left-to-right scan, and say why 12 must be one token;
 2. read a grammar in EBNF and draw, by hand, the tree it gives an expression;
 3. explain why * binds tighter than + using only the **shape** of the grammar, without a precedence table;
 4. turn each grammar rule into one method — a loop for (...)*, a call for a rule, an if for |;
 5. show why a right-recursive expr gets 1 - 2 - 3 wrong and a left-recursive one never stops, and which tests catch each;
 6. evaluate a tree by a post-order walk, and recognise that walk as postfix;
 7. measure the translator's time and its recursion depth, and say which one depends on the **shape** of the input.
-

Part 0 — Before you start**0.1 Environment**

From the DSA27 folder, with the virtual environment active (the prompt starts with (.venv)):

```
git pull
pytest -m "not challenge" -q          # environment check: must pass
pytest tests/test_stack_queue.py -q  # Weeks 6-7: must pass
pytest tests/test_translation.py -q  # 43 failed, 12 passed
```

The 12 that already pass are your Week 6 evaluate_postfix (11 tests) and test_to_edges_of_a_leaf which tests code given to you. If evaluate_postfix is not passing yet, finish it first (Lab 06): two of this week's end-to-end tests compare your calculator with it.

0.2 Prerequisites

- **Your Stack and infix_to_postfix** (Week 6). `evaluate_postfix` uses the stack, and `test_both_routes_agree` runs the whole shunting-yard route next to yours.
- **Recursion** (Lecture 03): the parser is three methods that call each other, and one of them calls back to the top.
- **Trees and post-order** (Lecture 11): the parser's result is a binary tree, and `evaluate` walks it children-first.

0.3 Read the skeleton: what is given, what is yours

Open `dsa/translation.py` and read it top to bottom before writing anything. The grammar is in a comment near the top; copy it onto paper, because every part of this lab refers to it:

```
expr  := term (('+' | '-' ) term)*
term  := factor (('*' | '/' ) factor)*
factor := NUMBER | '(' expr ')' | '-' factor
```

Given to you	Yours
<code>Num(value)</code> — a leaf; <code>BinOp(op, left, right)</code> — an operator node; both with <code>==</code> and a readable <code>repr</code>	<code>tokenize(text)</code>
<code>to_edges(node)</code> — pairs for <code>viz.draw.draw_tree</code>	<code>Parser.parse()</code>
<code>Parser.__init__</code> — the tokens in the course Array, and <code>position = 0</code>	<code>Parser.expr()</code> , <code>Parser.term()</code> , <code>Parser.factor()</code>
<code>Parser.peek()</code> — the current token, or <code>None</code> at the end	<code>evaluate(node)</code>
<code>Parser.advance()</code> — return the current token and move past it	<code>calculate(text)</code>

`peek` and `advance` are the only way your methods should touch the tokens. Decide by peeking; consume by advancing.

0.4 The storage rule, this week

The `Parser` is a structure, and it keeps its tokens in the course Array — that is done for you in `__init__`. The tree is made of **node objects** (`Num`, `BinOp`), which the rule allows. `tokenize` returns a Python list: lists are the interface for data passed in and out. Two rules follow from the point of the week:

- **No eval**, and no regular expressions (re). Python's `eval("3 + 4 * 2")` would pass most of the end-to-end tests and teach you nothing.
- **Do not build a precedence table** in the parser. Precedence must come from which method handles which operator — that is the lesson.

0.5 The tests, by stage

Every command is `pytest tests/test_translation.py -v -k "<filter>":`

Stage	-k filter	Tests
lexer	tokenize	13
parser	parse or preced or assoc or unary or nested	14
evaluator	evaluate and not postfix	4
end to end	calculate or routes	10
everything	(no filter)	55

The remaining 14 are the 12 postfix tests — 11 for `evaluate_postfix`, and one that pairs it with your `tokenize` — and the 2 for `to_edges`.

Part 1 — tokenize: characters into tokens

Lecture 15, “Stage 1: The Lexer”.

1.1 The idea

One pass from left to right with an index `i`. At each position look at **one** character and decide which of four cases it is: a space (skip it), an operator or a parenthesis (a token by itself), the start of a number (read the whole number), or anything else (an error). The lexer knows nothing about grammar: `tokenize("3 + + 4")` is `['3', '+', '+', '4']`, and it is the parser’s job to refuse it.

1.2 Draw it

Write the text in boxes, one character per box, with the index under each — like the lecture’s figure — and bracket the characters of each token (`_` marks a space):

```

text      1  2  _  *  _  (  3  .  5  +  4  )
index     0  1  2  3  4  5  6  7  8  9 10 11
tokens    [12]  ]      *      (  [ 3.5 ] + 4  )
```

When you meet the 1 at index 0, where does the number end? Index 2 — the first character that is neither a digit nor `..`. The token is `text[0:2]`.

Checkpoint 1. Give `tokenize(t)`, or the exception it raises and why, for:

- (a) `" (12+3.25)* 4 "`
- (b) `"7-"`
- (c) `"3,5"`
- (d) `"1..2"`
- (e) `"3 . 5"`

(f) "2(3)"

Which of them will the **parser** later reject, although the lexer accepts them?

1.3 Write it

1. Start with an empty result list and `i = 0`; loop while `i < len(text)`.
2. Read `ch = text[i]`.
3. A space (`ch.isspace()`): move on.
4. One of `+ - * / ( )`: append it as a token; move on.
5. A digit or `.`: remember where the number starts; move `i` forward **while there is a next character** and it is a digit or `.`, counting the points as you go. Cut the number out with one slice. If it has more than one point, or is just `.`, raise `ValueError`.
6. Anything else: raise `ValueError`, and put the character and its position in the message — you will be glad of it later.

Hints:

- Check digits with `ch in "0123456789"`, not `ch.isdigit()`: `isdigit` is also true for Arabic-Indic digits such as `३`, and those are not part of our language.
- In step 5, the while condition must test `i < len(text)` **first**. A number at the very end of the text is the normal case.

1.4 Test it

```
pytest tests/test_translation.py -v -k tokenize
```

Thirteen tests:

- eight cases of `test_tokenize`, from `""` to `"-5"`;
- `test_tokenize_keeps_multi_digit_numbers_whole`, on `"123 + 45"`;
- `test_tokenize_accepts_a_decimal_point`, on `"3.5 + 1"`;
- three cases of `test_tokenize_rejects_unknown_characters`: `$`, a letter, and `&`.

1.5 When it fails

Bug	What you see	Fix
each digit appended as its own token	3 of the 13 fail — At index 0 diff: <code>'1' != '12'</code> for <code>"12*(3+4)"</code> , and <code>['1', '2', '3', '+', '4', '5'] == ['123', '+', '45']</code>	keep reading while the next character is a digit or <code>.</code>
spaces appended as tokens	5 of the 13 fail: the lists contain <code>' '</code> entries	a space separates; it is never a token

Bug	What you see	Fix
no <code>i < len(text)</code> in the inner loop	<code>IndexError: string index out of range in 7 of the 13 — every case that ends with a number</code>	test the bound before reading <code>text[i]</code>
no final <code>else</code> (unknown characters skipped)	<code>Failed: DID NOT RAISE <class 'ValueError'> in the 3 rejection tests</code>	raise <code>ValueError</code>

Part 2 — The grammar on paper

Lecture 15, “Grammars”. Before any parser code, make sure you can do by hand what the parser will do.

2.1 The idea

To read an expression with the grammar, split it the way the rules say:

1. an `expr` is split at the `+` and `-` signs that are **not** inside parentheses — the pieces are terms, and the operators between them are applied **left to right**;
2. a term is split at `*` and `/` in the same way — the pieces are factors;
3. a factor is a number, a parenthesised `expr` (start again at 1 inside), or `-` followed by a factor, which becomes `0 - factor`.

Each split becomes a node of the tree, with the **last** operator of a chain at the top — so the first one is deepest and is evaluated first.

2.2 Draw it

For `10 - 4 - 3`: the `expr` splits into the terms `10`, `4`, `3` with `-` between them. Left to right: `10 - 4` first, then that result `- 3`. The tree, with each operator in brackets:

```

      [-]
     /  \
    [-]   3
   /  \
  10   4

```

Checkpoint 2. Draw the tree the grammar gives, and compute the value, for:

- (a) `2 * 3 + 4`
- (b) `2 * (3 + 4)`
- (c) `-2 * 3`
- (d) `8 / 2 / 2 * 3`

(e) $2 - (3 - 4)$

In (c), is the unary minus inside or outside the multiplication? Does it matter for the value here? Would it matter for $-2 - 3$?

You can check each tree once your parser works (Part 5): `draw_tree(to_edges(...))` draws exactly what your parser built.

Part 3 — The parser: parse, expr, term, factor

Lecture 15, “Stage 2: Recursive Descent”. Four short methods; write them all before you test, because they call each other and none of them can be tested alone.

3.1 The idea

One method per rule, translated line by line (the lecture’s table):

In the grammar	In the method
a rule’s name inside another rule	a call: <code>self.term()</code>
<code>(...)*</code>	a while loop that runs while <code>peek()</code> is one of the operators
<code>a \ b \ c</code>	<code>if / elif</code> on <code>peek()</code>
a quoted token such as <code>' '</code>	check <code>peek()</code> is that token, then <code>advance()</code>

The lecture shows `expr` in full. `term` is the same shape one level down.

3.2 Draw it

The parser’s calls form a tree, and at any moment the Python call stack holds one path of it. Indent one line per call:

```

expr
  term
    factor      reads 3
  term          (after expr reads '+')
    factor      reads 4
    factor      (after term reads '*') reads 2

```

That is $3 + 4 * 2$: two terms for the `expr`, and the second term has two factors.

Checkpoint 3. For $(1 + 2) * 3$:

- (a) Write the indented list of calls, as above.
- (b) How many calls of `expr`, `term` and `factor` are there in total?

- (c) How many of the three methods are on the call stack at the deepest point?
- (d) Which method, and which line of your plan, raises the `ValueError` for `"(3 + 4"`, for `"3 + 4 )"` and for `"()"`?

3.3 Write it

expr — as in the lecture: parse a term; while the next token is `+` or `-`, consume it, parse another term, and make a `BinOp` with the tree so far on the **left**. Return the tree.

term — the same, with `factor` in place of `term` and `*` / `/` in place of `+` `-`.

factor — peek at the token:

1. None: the input ended where a factor was needed — `ValueError`.
2. `(`: consume it, parse an `expr`, then require `)` — if the next token is not `)`, `ValueError`; if it is, **consume it**. Return the inner tree.
3. `-`: consume it; return `BinOp("-", Num(0.0), <a factor>)`.
4. A number: consume it; return `Num(float(token))`.
5. Anything else: `ValueError`.

parse — refuse an empty token list; parse an `expr`; then, if any token is left, `ValueError`. Return the tree.

Hints:

- `self.peek()` in `("+", "-")` is safe at the end: `None` in `(...)` is `False`.
- Test the `None` case in `factor` **first**; the other tests look inside the token, and `None` has nothing to look at.
- A number token is recognisable by its first character: a digit or a `.`

3.4 Test it

```
pytest tests/test_translation.py -k "parse or preced or assoc or unary or nested"
```

Fourteen tests: a single number and `3 + 4`; `test_precedence_is_the_shape_of_the_tree` (`3 + 4 * 2`); parentheses; the two left-associativity tests (`1 - 2 - 3` and `8 / 4 / 2`); unary minus (`-5`); `((7))`; and six malformed inputs that must raise `ValueError`: `"`, `"3 +"`, `"(3 + 4"`, `"3 + 4 )"`, `"* 3"`, `"()"`.

The assertion messages print whole trees. Read them as nested calls: `BinOp('-', Num(1.0), BinOp('-', Num(2.0), Num(3.0)))` is `1 - (2 - 3)`.

3.5 When it fails

Bug	What you see	Fix
if instead of while in <code>expr</code>	<code>ValueError: unexpected '-' after the expression in the 1 - 2 - 3 test — one operator was read, the second left over</code>	a chain is a loop
<code>expr</code> calls <code>factor</code> (and handles all four operators)	the precedence test fails: <code>3 + 4 * 2</code> is parsed as <code>(3 + 4) * 2</code>	<code>expr</code> calls <code>term</code> ; <code>term</code> calls <code>factor</code>
) checked but not consumed	<code>ValueError: unexpected ')' after the expression in the parentheses and ((7)) tests</code>	<code>advance()</code> past the)
no check for leftover tokens in <code>parse</code>	<code>Failed: DID NOT RAISE <class 'ValueError'> for "3 + 4)"</code>	after <code>expr</code> , <code>peek()</code> must be <code>None</code>
no <code>None</code> check in <code>factor</code>	<code>TypeError: 'NoneType' object is not subscriptable for "3 +"</code>	check <code>None</code> first
<code>Num(token)</code> instead of <code>Num(float(token))</code>	<code>assert Num('3') == Num(3.0) — a string in the tree</code>	convert with <code>float</code>

Two bugs pass all 14 of these tests and show up only later, in Part 5: unary minus that calls `self.expr()` instead of `self.factor()`, and a forgotten empty-input check in `parse` (which is harmless, because `factor` then raises on the empty input anyway). Keep reading.

Part 4 — Break it: two grammars that look right

Lecture 15, “Two grammars that look right and are not”. Your parser passes its 14 tests. Save your `expr` somewhere safe (or rely on `git diff`), then try two tempting alternatives. Each is what you get by turning the **recursive** BNF form of the grammar straight into code.

4.1 Right recursion

Replace the body of `expr` with the right-recursive version from the lecture: parse a `term`; **if** the next token is `+` or `-`, return a `BinOp` whose right child is a recursive call to `self.expr()`.

Checkpoint 4. Before you run anything:

(a) What tree does it build for `1 - 2 - 3`, and what value does that give?

- (b) Of all 55 tests, which fail? (Hint: which test expressions have two + or - **at the same level**, one after the other?)
- (c) Why does $8 / 4 / 2$ still pass?

Run the whole file and compare:

```
pytest tests/test_translation.py -q
```

Read the failing assertions. A wrong answer with no exception is the worst kind of bug; the only thing that caught it is a test written for exactly this case.

4.2 Left recursion

Now write `expr` as the textbook grammar `expr := expr ('+' | '-') term | term` reads: the first thing `expr` does is call `self.expr()`.

```
pytest tests/test_translation.py -q -x
```

`-x` stops at the first failure. You will see `RecursionError: maximum recursion depth exceeded`, on the very first parser test — “3”, a single number. Why? `expr` calls itself before it has consumed a token, so every call starts from the same position, and nothing ever gets closer to a base case. (Without `-x`, 24 tests fail this way: every test that parses anything.)

Put your loop back and check: 14 passed on the parser filter.

Part 5 — evaluate and calculate

Lecture 15, “Stage 3: Walking the Tree”. (At home, if the session runs out.)

5.1 The idea

`evaluate(node)` is a post-order walk: a `Num` is its own value; a `BinOp` is worth “left value op right value”, which needs both children’s values **first**. Four operators; anything else is a `ValueError`. Division by zero needs no code of yours: Python’s `/` raises `ZeroDivisionError` itself.

`calculate(text)` joins the three stages: tokenize, parse, evaluate.

5.2 Draw it

Number the nodes of the tree in the order `evaluate` **finishes** them, and write each node’s value beside it — the lecture’s figure for $(3 + 4) * (5 - 2)$.

Checkpoint 5. For $(8 - 2) / (1 + 2)$:

- (a) Draw the tree, and number the nodes in the order `evaluate` finishes them.
- (b) Write the node labels in that order. What is this sequence called, and where did you

meet it before?

(c) What value does calculate return, and what type is it?

5.3 Write it and test it

```
pytest tests/test_translation.py -v -k "evaluate and not postfix"
pytest tests/test_translation.py -v -k "calculate or routes"
```

Four evaluator tests (a leaf, a hand-built tree, division by zero, an unknown operator ^), then ten end-to-end: nine calculate cases and test_both_routes_agree, which compares your calculator with evaluate_postfix(infix_to_postfix(tokens)) on four expressions.

5.4 When it fails

Bug	What you see	Fix
right - left (operands swapped)	the evaluator tests pass — they use only + and * — then calculate("-5 + 2") gives 7.0, "1 - 2 - 3" gives 2.0; 4 tests fail	left operand first
no recursion: node.left.value	AttributeError: 'BinOp' object has no attribute 'value' in 8 tests	call evaluate on each child
unknown operator returns None // instead of /	Failed: DID NOT RAISE <class 'ValueError'> calculate("10 / 4") gives 2.0, not 2.5	raise ValueError after the four cases true division
unary minus built with self.expr() in factor	only calculate("-5 + 2") fails: -7.0 instead of -3.0 — the minus swallowed 5 + 2	the operand of unary minus is a factor

The first and last rows are why end-to-end tests exist: each stage's own tests passed, and the calculator was still wrong.

When all 55 pass, run the stages you did not write by hand against each other:

```
from dsa.translation import tokenize, Parser, evaluate
from dsa.translation import evaluate_postfix
from dsa.stack import infix_to_postfix
text = "2 * (3 + (4 - 1))"
print(evaluate(Parser(tokenize(text)).parse()))      # 12.0
print(evaluate_postfix(infix_to_postfix(tokenize(text)))) # 12
```

The tree route gives a float (every Num holds one); the stack route keeps integers when it can. Equal values, different types — which is why `test_both_routes_agree` compares with `pytest.approx`.

Part 6 — See it

6.1 Draw the tree your parser built

In `notebooks/15-language-translation.ipynb`:

```
from viz.draw import draw_tree
from dsa.translation import Parser, tokenize, to_edges

def show(text):
    return draw_tree(to_edges(Parser(tokenize(text)).parse()), title=text)

show("3 + 4 * 2")
show("(3 + 4) * 2")
show("1 - 2 - 3 + 4")
```

Compare each picture with your answers to Checkpoint 2. A drawing is the fastest way to find a precedence or associativity bug: a wrong tree is obvious at a glance, where a wrong number is not.

6.2 Watch the call stack

Make the parser print its own calls — in a **subclass**, so that `dsa/translation.py` stays clean:

```
from dsa.translation import Parser, tokenize

class Tracing(Parser):
    depth = 0

    def expr(self):
        return self._traced("expr", super().expr)

    def term(self):
        return self._traced("term", super().term)

    def factor(self):
        return self._traced("factor", super().factor)

    def _traced(self, name, method):
        indent = " " * Tracing.depth
```

```

print(f"{indent}{name} at {self.position} ({self.peek()!r})")
Tracing.depth += 1
try:
    return method()
finally:
    Tracing.depth -= 1

```

```
Tracing(tokenize("(1 + 2) * 3")).parse()
```

Check the output against your answer to Checkpoint 3. The indentation is the depth of Python's call stack: the parser has no stack of its own, because the call stack **is** its stack (Lecture 15, "The calls are the parse tree").

Part 7 — Measure it

Lecture 15, "Measured". Two questions: how does the **time** grow with the length of the input, and how does the **recursion depth** grow with its shape?

7.1 Time

```

import random
from viz.complexity import measure, plot_growth
from dsa.translation import calculate

def balanced(n, rng=random.Random(15)):
    """A balanced, fully parenthesised expression with n numbers."""
    parts = [str(rng.randint(1, 9)) for _ in range(n)]
    while len(parts) > 1:
        merged = [f"({parts[i]} {rng.choice('+-*')} {parts[i + 1]})"
                  for i in range(0, len(parts) - 1, 2)]
        if len(parts) % 2:
            merged.append(parts[-1])
        parts = merged
    return parts[0]

sizes = [500, 1000, 2000, 4000, 8000]
result = measure(calculate, sizes, balanced)
plot_growth({"calculate": result}, reference=["n"], loglog=True)

```

measure builds each input outside the timing and keeps the best of three runs. Expect a straight line of slope 1: double the numbers, double the time — $O(n)$. Your times will differ from anyone else's; the slope will not.

7.2 Depth

```
def deepest(make):
    k = 1
    while True:
        try:
            calculate(make(k))
        except RecursionError:
            return k - 1
        k += 1

print(deepest(lambda k: "(" * k + "1" + ")" * k)) # nested
print(deepest(lambda k: " + ".join(["1"] * k))) # flat
```

Checkpoint 6. Predict before you run:

- (a) How many Python frames does one pair of parentheses cost the parser? Roughly how many pairs will fit under Python's limit of 1,000 frames?
- (b) The flat chain $1 + 1 + \dots + 1$ is parsed with a **loop**. Does it fail at all? If it does, which function runs out of stack, and at about what length?
- (c) How deep does a **balanced** expression with 1,000 numbers go?

The chain takes a few seconds, because it parses every length from 1 upwards. Run from a notebook, both answers come out a little smaller than from a script: Jupyter itself is already using some frames when your code starts.

Part 8 — Exercises at a glance

Method	Target cost	The trap	Tests (Part 0.5)
tokenize	$O(n)$	numbers stay whole; $i < \text{len}(\text{text})$ first; unknown character raises	lexer (13)
Parser.expr	$O(n)$ overall	a loop ; the old tree goes on the left ; calls term	parser (14)
Parser.term		the same shape; calls factor	parser
Parser.factor		None first; consume the <code>)</code> ; unary minus calls factor	parser
Parser.parse		empty input; tokens left over	parser
evaluate	$O(n)$	children first; swapped operands pass the unit tests; unknown operator	evaluator (4)
calculate	$O(n)$	three lines	end to end (10)

All 55 at once:

```
pytest tests/test_translation.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that nothing in `dsa/translation.py` calls `eval`, imports `re`, or keeps a table of precedences — the tests cannot see those, but the TA will look.

Part 9 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week15-questions.md`, with answers in `week15-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week15.py`: `to_postfix`, `to_infix`, `to_infix_minimal`, `postfix_to_tree` and `calculate_with_power` (W15-C1 to W15-C5):

```
pytest tests/test_practice_week15.py -v
```

The round-trip tests of C3 parse your output with **your** Parser, so finish this lab first. Hint for `to_infix_minimal`: a child needs parentheses when its operator binds more loosely than its parent's — or equally loosely, if it is the **right** child.

2. **W15-S1** — draw the trees of four expressions, and of the right-recursive parser's mistake.
3. **W15-T2** — the call trace of `2 * (3 - 1) + 4`, with the tokens each call consumes.
4. **W15-B1** — the right-recursive `expr` of Part 4.1, as an exam question: find an input on which it fails without running it.
5. **W15-K2** — the recursion depth of the parser and of `evaluate`, for nested parentheses and for a flat chain. Check against Part 7.2.

The worked solutions are in `solutions/dsa/translation.py` and `solutions/practice/week15.py` — for after you have tried. `pytest --solutions tests/test_practice_week15.py` runs the tests on them.

Part 10 — Course wrap-up and exam preparation

This is the last lab. There is no next week to bridge to — so instead, a look back at what your repository now contains, and a plan for the final.

10.1 What you built

Run the whole suite once:

```
pytest -q
```

Every module that passes is a structure or algorithm you wrote yourself, on the course Array, with its cost stated in its docstring and checked by a test. Look at the list of files in `dsa/` and, for each, say aloud one sentence: what it keeps, and what its main operation costs. If you cannot, that is the week to revise first.

This week's calculator used the Array (the parser's tokens), recursion (every parser method, and `evaluate`), your Stack on your `DynamicArray` (the postfix route), node objects (`Num`, `BinOp`) and a tree walk (post-order). One program, most of the course.

10.2 The final exam

- **Format:** 30 marks multiple choice (one answer of four), 30 marks written — traces, drawings of a structure after each operation, complexity with a justification, find-and-fix, and short code. On paper, with no computer.
- **To pass the course:** at least 60% overall **and** at least 30% of the final exam; attendance of at least 75% to sit it. See the course guide, `docs/course/00-course-guide.md`.
- **Examinable:** every week, 1 to 15. Dynamic programming is enrichment and is not examined.

10.3 How to prepare

1. **The question banks, closed.** For each week, answer the questions on paper with the answers file shut, then mark yourself against it. For every multiple-choice question you got wrong — or right for the wrong reason — read the explanation of **every** option.
2. **The mock papers**, in `docs/question-bank/`, under exam conditions: one sitting, closed book, no computer.
3. **Draw.** For each structure — dynamic array, linked list, stack, ring buffer, BST, heap, hash table, graph, and now the expression tree — draw it after each operation of a short sequence. The written paper rewards pictures that are exactly right.
4. **Write from memory.** Pick one function a day from `dsa/` — `binary_search`, `sift_down`, `partition`, `bfs`, `Parser.expr` — and write it on paper without looking. Then run it. What you cannot write on paper, you do not yet know.
5. **Complexity, with a reason.** For every function, say its cost **and why**: which loop, which recurrence, which amortised argument. The exam leans on **[why]** questions.

Where it goes from here depends on your program: SWE students meet grammars and parsing again, formally, in **SWE141 Software Construction**; AI students meet algorithm design in **AI3001**. Everyone meets these structures in every program they will ever read.

Summary

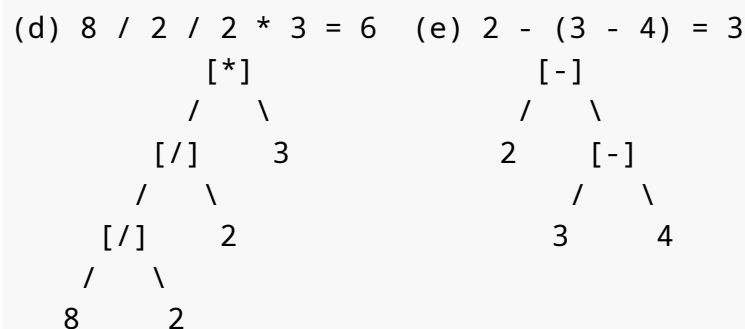
Idea	The one line to keep
Three stages	Lexer: text $\rightarrow$ tokens. Parser: tokens $\rightarrow$ tree. Evaluator: tree $\rightarrow$ value.
Lexer	One scan; numbers stay whole; a space separates; unknown characters raise.
Grammar	expr of terms, term of factors; factor recurses to expr through (.
Precedence	The lower the rule, the tighter the operator: the shape , not a table.
Recursive descent	One method per rule; (...) [*] is a while; decide by peek().
Associativity	The old tree becomes the left child.
Right recursion	Builds 1 - (2 - 3): wrong, silently — 3 tests catch it.
Left recursion	Calls itself before consuming a token: RecursionError.
Errors	Empty input, a missing), a token left over: ValueError.
Evaluate	Post-order: children first. The post-order sequence is postfix.
Measured	Time $O(n)$ in the length; depth grows with the height of the tree.

Answers to the checkpoints

Checkpoint 1. (a) ['(', '12', '+', '3.25', ')', '*', '4'] — spaces anywhere, including at both ends, just vanish. (b) ['7', '-']. (c) ValueError: unexpected character ',' at position 1 — a comma is not a decimal point in our language. (d) ValueError: malformed number '1..2' — the scan reads 1..2 as one number and then finds two points. (e) ValueError: malformed number '.' — the spaces split 3 . 5 into 3, a lone ., and 5, and the lone . is not a number. (f) ['2', '(', '3', ')']. The lexer accepts (b) and (f), but the parser rejects both: 7 - needs a factor after the - (unexpected end of input), and 2(3) has no operator between 2 and ((unexpected '(' after the expression). (a) parses, and its value is 61.

Checkpoint 2.

(a) $2 * 3 + 4 = 10$	(b) $2 * (3 + 4) = 14$	(c) $-2 * 3 = -6$
$ \begin{array}{c} [+] \\ / \quad \backslash \\ [*] \quad 4 \\ / \quad \backslash \\ 2 \quad 3 \end{array} $	$ \begin{array}{c} [*] \\ / \quad \backslash \\ 2 \quad [+] \\ / \quad \backslash \\ 3 \quad 4 \end{array} $	$ \begin{array}{c} [*] \\ / \quad \backslash \\ [-] \quad 3 \\ / \quad \backslash \\ 0 \quad 2 \end{array} $



In (c) the minus is **inside**: unary minus is a factor, so -2 is complete before term sees the $*$ — the tree is $(0 - 2) * 3$. For the value it makes no difference here, since $(-2) \times 3 = -(2 \times 3)$. For $-2 - 3$ it would: $(0 - 2) - 3 = -5$, which is right; reading the minus as covering the whole rest, $-(2 - 3) = 1$, would be wrong. In (d) all three operators are at the same level (term), so they apply left to right: $((8 / 2) / 2) * 3$.

Checkpoint 3. (a)

```

expr
  term
    factor      reads '(' ...
    expr
      term
        factor   reads 1
        term     (after expr reads '+')
        factor   reads 2
                ... and ')'
    factor      (after term reads '*') reads 3
  
```

(b) 9 calls: 2 of expr, 3 of term, 4 of factor.

(c) 6: expr, term, factor, then expr, term, factor again inside the parentheses — three per level of nesting, plus parse below them, which calls the first expr.

(d) " $(3 + 4$ ": factor, after its inner expr returns and the next token is None instead of $)$ — "missing $)$ ". " $3 + 4)$ ": parse, after expr returns with the $)$ still unread — a token left over. " $()$ ": factor again, but the inner one: the $($ case calls expr, expr calls term, term calls factor, and that factor finds $)$, which cannot start a factor.

Checkpoint 4. (a) `BinOp('-', Num(1.0), BinOp('-', Num(2.0), Num(3.0)))`, that is $1 - (2 - 3) = 2$, where the right answer is -4 . (b) Exactly 3 of the 55: `test_subtraction_is_left_associative`, `test_calculate[1 - 2 - 3--4.0]` (`assert 2.0 == -4.0 ± 4.0e-06`) and `test_both_routes_agree`. $1 - 2 - 3$ is the only test expression with two $+/-$ in a row at the same level — $2 * (3 + (4 - 1))$ has two, but at different levels of parentheses — and a chain of additions would give the right answer either way. (c) Because $/$ is handled by term, and your term still uses a loop. The same right-recursive bug in term would fail $8 / 4 / 2$ (giving 4, not 1).

Checkpoint 5. (a) The root is $/$; its left child is $-$ over 8 and 2, its right child $+$ over 1 and 2. Finishing order: 8 (1), 2 (2), $-$ (3, value 6), 1 (4), 2 (5), $+$ (6, value 3), $/$ (7, value 2). (b) $8 \ 2 \ - \ 1$

2 + / — the **postfix** form of the expression, from Lecture 06. The post-order walk of the tree and the postfix list are the same sequence. (c) 2.0, a float: every Num holds a float, and / is true division.

Checkpoint 6. (a) Three frames per pair — one each for `expr`, `term` and `factor`. With about 1,000 frames available, about 330 pairs: run as a script, the last depth that works is **329**. (b) Yes. The parser's loop handles any length, but the tree it builds leans left all the way down, its height is one less than the number of 1s, and the **recursive evaluate** goes one frame per level. From a script the longest chain that works has **997** numbers. (c) Only about 40 frames in all — measured, the deepest point of `calculate(balanced(1000))` is 40. A balanced tree of 1,000 leaves has height $\lceil \log_2 1000 \rceil = 10$; the parser uses three frames per level of parentheses, 30 in all, and `calculate`, `parse`, `peek` and `advance` add the rest. Time depends on the length; depth depends on the shape.